

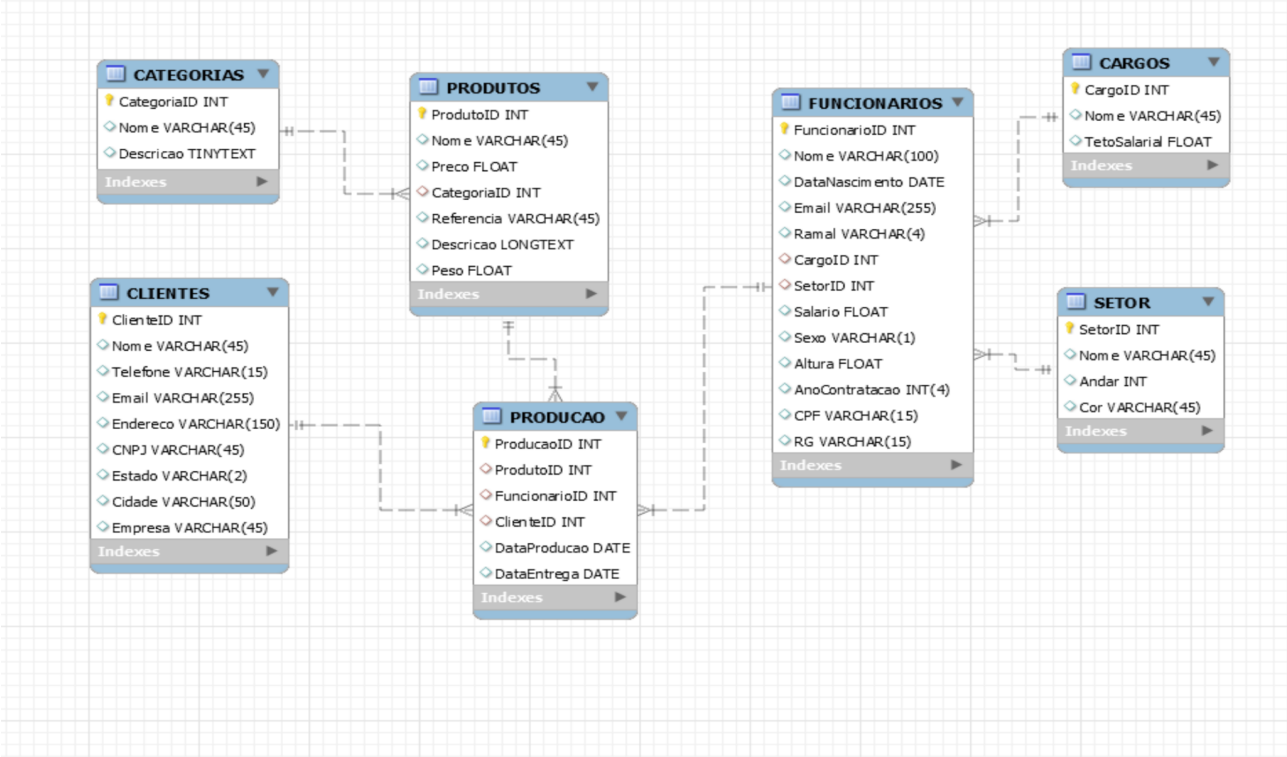
PHP Orientado a Objetos com Banco de Dados

Material explicativo para demonstrar classes, objetos, construtores, métodos, parâmetros, atributos, encapsulamento, herança, polimorfismo e abstração usando a base EER de produção, produtos, clientes, funcionários, cargos, setores e categorias.

Objetivo didático

O objetivo é sair de uma leitura do modelo EER para uma aplicação navegável. O aluno deve compreender que cada tabela pode inspirar uma classe de domínio, enquanto o acesso ao banco pode ser isolado em uma classe Database e em classes Repository.

Modelo EER usado como base



O EER indica relações centrais: produtos pertencem a categorias; funcionários pertencem a cargos e setores; a produção conecta produtos, funcionários e clientes. Essas relações orientam as chaves estrangeiras e as telas de cadastro.

Mapeamento conceitual entre banco e OOP

Banco de dados	PHP OOP	Uso didático
Tabela produtos	class Produto	Objeto com nome, preço, referência, descrição e peso.
Tabela clientes	class Cliente extends Pessoa	Herança a partir de campos comuns de pessoa.
Tabela funcionários	class Funcionario extends Pessoa	Métodos como salarioAnual() demonstram comportamento.
Classe Database	Singleton + PDO	Centraliza conexão, charset, erros e segurança.
Repository	GenericRepository	CRUD com prepared statements e baixo acoplamento.

Classe e objeto

Classe é o molde. Objeto é uma instância em memória. No projeto, Produto é a classe; cada registro vindo da tabela produtos pode virar um objeto Produto.

Constructor

O construtor __construct(array $dados) recebe os dados do banco ou do formulário e hidrata o objeto. Isso evita espalhar atribuições repetidas pela aplicação.

Atributos

Os atributos representam estado. Na solução do professor, os dados ficam encapsulados no array protegido $dados dentro de AbstractEntity.

Métodos e parâmetros

Métodos representam comportamento. Produto::calcularFreteEstimado(float $valorPorKg) recebe um parâmetro e calcula um resultado usando o peso do produto.

Encapsulamento

A aplicação evita acesso direto livre aos dados internos e usa get(), set(), toArray() e métodos específicos. Isso melhora validação, manutenção e segurança.

Herança

Pessoa concentra comportamento comum de pessoas. Cliente e Funcionario herdam de Pessoa e especializam seus resumos.

Polimorfismo

Produto, Cliente, Funcionario e Producao implementam resumo(), mas cada classe responde de forma própria. O mesmo código pode percorrer objetos diferentes e chamar o mesmo método.

Abstração

AbstractEntity, GenericRepository e RepositorioInterface escondem detalhes repetitivos e deixam a regra de negócio mais limpa.

Exemplo de código principal

```
abstract class AbstractEntity implements ResumoApresentavel {
protected array $dados = [];
public function __construct(array $dados = []) { $this->fill($dados); }
public function get(string $atributo, mixed $padrao = null): mixed {
return $this->dados[$atributo] ?? $padrao;
}
abstract public static function tableName(): string;
abstract public static function primaryKey(): string;
}
```

Roteiro de aula recomendado

Etapa	Tema	O que fazer
1	Importar banco	Executar db/02_database_demo_corrigido.sql e validar tabelas.
2	Ler EER	Identificar entidades, chaves primárias e relacionamentos.
3	Criar Database	Conectar com PDO e prepared statements.
4	Criar entidades	Implementar Produto, Cliente, Funcionario e Producao.
5	Criar Repository	Separar SQL do restante da aplicação.
6	Criar telas	Listar, cadastrar, editar, excluir e visualizar registros.
7	Comparar versões	Analisar PDO/OOP versus MySQLi procedural.

Critérios de aceite da aplicação

• Conectar ao banco demo via PDO na versão principal.

• Navegar por todos os módulos principais.

• Criar, editar, listar, visualizar e excluir registros.

• Demonstrar no código todos os conceitos OOP solicitados.

• Manter SQL dentro da pasta db e usar a imagem EER como referência visual.